

DataSplits.jl: Data splitting strategies for machine learning in Julia

Davide Crucitti^{1,2}, Carlos Pérez Míguez¹, and Adrián Mosquera Orgueira^{1,3}

¹ Group of Computational Genomics and Hematology (GrHeCoXen), Health Research Institute of Santiago de Compostela (IDIS), Santiago de Compostela, Spain ² Departamento de Farmacología, Farmacia y Tecnología Farmacéutica, Universidade de Santiago de Compostela, Santiago de Compostela, Spain ³ Division of Hematology, Complexo Hospitalario Universitario de Santiago de Compostela, SERGAS, Santiago de Compostela, Spain

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Matt Graham

Reviewers:

- @MohamedLaghdafHABIBOULLAH
- @jonocarroll

Submitted: 02 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

DataSplits.jl is a Julia package (Bezanson et al., 2017) for constructing training, validation, test, and cross-validation partitions in scientific machine learning workflows. While random train/test splits and standard k-fold cross-validation are widely available in machine learning libraries, many scientific applications require more controlled splitting procedures. The package exposes a single entry point:

```
res = partition(data, strategy; kwargs...)
```

The returned split objects expose train, validation, and test indices, lazy observation views, materialised subsets, and row-pair representations suitable for external model-evaluation frameworks. Supported strategy families include representative sample selection for calibration modelling, distance-based selection in spectral or molecular descriptor spaces, group-aware validation to avoid leakage between related observations, and time-aware validation for temporally ordered data. The package is designed to be lightweight and interoperable with the Julia machine learning ecosystem, including MLUtils.jl, Flux.jl, and MLJ.jl. It can be installed from the Julia General Registry with `add DataSplits`. Source code is available at <https://github.com/davide-grheco/DataSplits.jl> and documentation at <https://davide-grheco.github.io/DataSplits.jl>.

Statement of need

The choice of data partitioning strategy can strongly affect the reliability of model evaluation, particularly in small-data scientific domains such as chemometrics, spectroscopy, metabolomics, materials informatics, and quantitative structure–property (QSPR) or structure–activity (QSAR) modelling. In these settings, independent and identically distributed (i.i.d.) random splits may be inappropriate: samples can be highly structured, class distributions may be imbalanced, repeated measurements may induce group leakage, and calibration sets may need to span the relevant feature and response spaces (Kohavi, 1995). Representative sample selection algorithms such as Kennard–Stone/CADEX (Kennard & Stone, 1969) and SPXY/MDKS (Galvão et al., 2005; Saptoro et al., 2012) are commonly used in chemometrics to construct calibration and validation sets with broad coverage of the experimental domain. Related approaches include the Duplex algorithm (Snee, 1977), which selects both training and test sets simultaneously using the maximin criterion; OptiSim and dissimilarity-based selection methods (Clark, 1997); the Onion method (Gallagher & O’Sullivan, 2020) and its joint X–y extension, XY-Onion (Ezenarro & others, 2025); electrostatic field-strength selection (He et

41 [al., 2026](#)); spectral clustering-based splits; and classification-oriented adaptations such as
42 the Morais–Lima–Martin algorithm ([Lelis Medeiros de Morais et al., 2019](#)). Time-series and
43 financial machine learning workflows call for chronological splitting strategies such as purged
44 k-fold cross-validation and combinatorial purged cross-validation ([López de Prado, 2018](#)),
45 and blocked cross-validation ([Bergmeir & Benítez, 2012](#); [Roberts et al., 2017](#)). Target-aware
46 cross-validation strategies such as Venetian blinds ([Næs et al., 2002](#)) assign samples sorted
47 by response value across folds in round-robin order, guaranteeing uniform target coverage
48 without binning. These methods are useful in scientific modelling but, to our knowledge, are
49 not currently available together in any general-purpose Julia splitting library.

50 DataSplits.jl addresses this gap by collecting these splitting strategies in a single Julia package
51 with a consistent interface. It is intended for researchers who need reproducible, inspectable,
52 and domain-appropriate partitions while remaining compatible with existing Julia modelling
53 tools.

54 State of the field

55 The closest Python equivalents are scikit-learn ([Pedregosa et al., 2018](#)) and astartes ([Burns
56 et al., 2023](#)). Scikit-learn provides group-aware and time-series CV strategies (GroupKFold,
57 TimeSeriesSplit, StratifiedGroupKFold) but does not implement distance-based selection
58 methods such as Kennard–Stone, SPXY, or OptiSim. Astartes fills that gap for molecular and
59 materials data, offering Kennard–Stone and several dissimilarity-based splitters, but does not
60 cover group-aware CV, time-series CV, or nested cross-validation. DataSplits.jl covers both
61 families — distance-based selection and a broad catalogue of standard CV strategies — in a
62 consistent interface.

63 Within Julia, MLUtils.jl provides an observation interface based on numobs, getobs, and lazy
64 observation views, together with data loaders and basic splitting utilities ([JuliaML, 2022](#)).
65 Flux.jl re-exports MLUtils data-loading functionality for neural network training ([Innes, 2018](#)).
66 MLJ.jl provides a comprehensive model-evaluation framework with built-in holdout and cross-
67 validation resampling strategies, and also accepts explicit train/test row-pair vectors for custom
68 resampling workflows ([Blaom et al., 2020](#)). To our knowledge, none of these packages
69 implement distance-based, group-stratified, or purged time-series splitting strategies.

70 DataSplits.jl is not intended to replace these packages. Instead, it complements them
71 by focusing on splitting strategies that are especially useful in scientific datasets and
72 calibration/validation design. The output of these strategies can be consumed by other
73 packages: lazy views can be passed to Flux/MLUtils data loaders, while explicit row pairs can
74 be supplied to MLJ evaluation routines without requiring DataSplits.jl to depend on MLJ.jl.

75 DataSplits.jl was developed as a separate package rather than an extension to MLJ.jl or
76 MLUtils.jl because its core contribution is split construction rather than model evaluation or
77 data loading. Keeping the package lightweight allows the same split definitions to be reused
78 across MLJ, Flux, MLUtils, and custom scientific workflows without imposing a dependency
79 on any specific modelling framework.

80 Software design

81 The central abstraction in DataSplits.jl is a splitting strategy. Each strategy is represented
82 by a concrete subtype of AbstractSplitStrategy, while cross-validation strategies subtype
83 AbstractCVStrategy. Users call partition(data, strategy; kwargs...) to produce an
84 AbstractSplitResult. A single entry point is preferable to separate per-algorithm functions
85 because it lets user code switch strategies without changing call sites. Result types include
86 two-way train/test splits, three-way train/validation/test splits, and cross-validation splits
87 containing one split per fold.

88 The package exposes stable accessors such as `trainindices`, `testindices`, `valindices`, and
 89 folds, so downstream code does not need to depend on internal result fields. Exposing indices
 90 rather than only data subsets matters in practice: the same split can be applied to multiple
 91 aligned arrays (features, labels, metadata, auxiliary covariates) without re-running the selection
 92 algorithm. For data extraction, `DataSplits.jl` provides both lazy and materialised interfaces.
 93 Functions such as `trainview`, `testview`, `valview`, and `splitview` use `MLUtils`-compatible
 94 observation views, while `traindata`, `testdata`, `valdata`, and `splitdata` materialise subsets
 95 using `getobs`.

96 All distance-based strategies have a lazy counterpart (e.g. `LazyKennardStoneSplit`,
 97 `LazyOptiSimSplit`) that computes distances on-the-fly in $O(N)$ peak memory instead of
 98 precomputing the full $O(N^2)$ distance matrix. This makes the strategies applicable to datasets
 99 where the full matrix would not fit in RAM, at the cost of increased runtime — a trade-off
 100 that is important for the high-dimensional descriptor matrices common in chemometrics and
 101 molecular modelling.

102 The package is designed to be extensible. Adding a custom strategy requires subtyping
 103 `AbstractSplitStrategy`, declaring which auxiliary slots (`:data`, `:target`, `:time`, `:groups`)
 104 the strategy reads via the `consumes` trait, and implementing a single `_partition` method.
 105 The partition entry point handles cohort-size resolution, slot validation, and feature-matrix
 106 conversion automatically.

107 The time-aware strategies include `BlockedCV`, which implements blocked cross-validation
 108 ([Bergmeir & Benítez, 2012](#)) by partitioning the time axis into contiguous blocks to avoid
 109 look-ahead bias, and `PurgedKfold`, which implements purged k-fold cross-validation ([López de
 110 Prado, 2018](#)) by additionally removing observations from the training fold whose labels overlap
 111 in time with the test fold.

112 A key design choice is to keep ecosystem integration lightweight. `DataSplits.jl` provides
 113 `rowpairs`, which converts any split result into the vector of (`train_indices`, `test_indices`)
 114 pairs accepted by MLJ's `evaluate!` `resampling=` keyword, without requiring a dependency
 115 on `MLJ.jl` or `MLJBase.jl`. This avoids coupling split construction to any particular modelling
 116 framework.

117 The test suite checks split-size resolution, non-overlap between cohorts, complete fold coverage
 118 where applicable, reproducibility under fixed random number generators, stratification and
 119 group constraints, and consistency of lazy and materialised data extraction.

120 Example usage

121 A representative train/test split can be constructed using the Kennard–Stone strategy:

```
using DataSplits
```

```
res = partition(X, KennardStoneSplit(); train = 0.8, test = 0.2)
```

```
X_train, X_test = splitview(res, X)
```

```
train_idx = trainindices(res)
```

```
test_idx = testindices(res)
```

122 For stratified cross-validation with MLJ, the result can be converted to row pairs directly:

```
using DataSplits, MLJ, Random
```

```
cvs = partition(X, StratifiedKfold(5); target = y, rng = Xoshiro(42))
```

```
mach = machine(model, X, y)
```

```
evaluate!(mach; resampling = rowpairs(cvs), measure = accuracy)
```

123 The same split indices can also be applied to multiple aligned data sources, enabling integration
124 with Flux data loaders:

using DataSplits, Flux

```
res = partition(X, KennardStoneSplit(); train = 0.8, test = 0.2)
loader = Flux.DataLoader(trainview(res, X, y); batchsize = 64, shuffle = true)
```

125 In chemometric calibration, Kennard–Stone-style selection constructs a training set that spans
126 the observed descriptor space, reducing the risk that validation samples fall outside the
127 calibration domain — a problem that random splits cannot guard against by design. The
128 practical difference is illustrated in Figure 1.

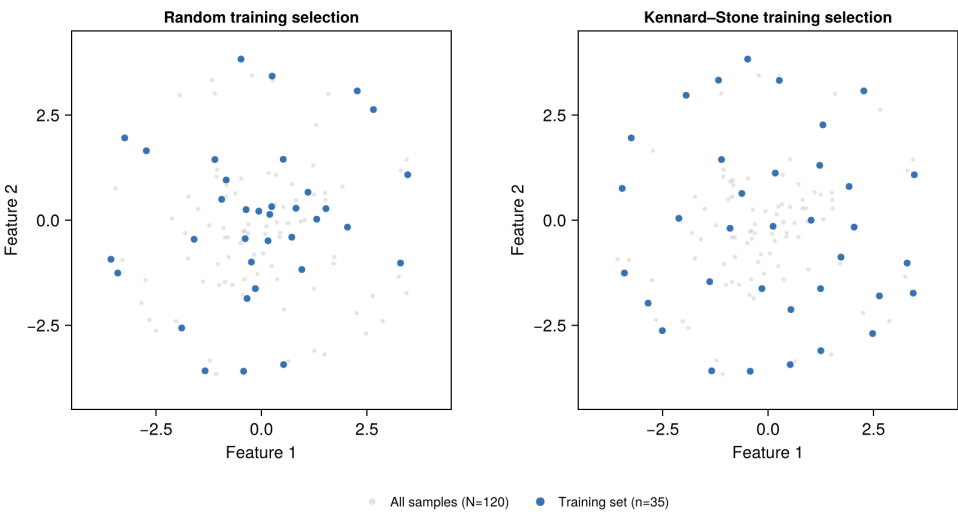


Figure 1: Comparison of random and Kennard–Stone training-set selection on a synthetic dataset with a dense central region and a sparse outer boundary. Gray points denote all samples (N=120) and blue points the selected training set (n=35). Random selection follows the empirical density and concentrates training samples in the center, whereas Kennard–Stone produces a more space-filling selection that better covers the boundary of the experimental domain.

Strategy family	Examples	Typical use
Distance-based	KennardStoneSplit, SPXYSplit, OptiSimSplit, DuplexSplit, OnionSplit, FieldStrengthSplit, SpectralSplit	Calibration/test design in chemometrics and descriptor spaces
Target-aware	TargetPropertySplit, StratifiedKFold, StratifiedShuffleSplit, XYOnionSplit, VenetianBlindsCV	Class-balanced or response-aware validation
Group-aware	GroupShuffleSplit, GroupKFold, StratifiedGroupKFold	Avoiding leakage across related observations
Time-aware	TimeSplit, BlockedCV, PurgedKFold, CombinatorialPurgedKFold	Temporally ordered validation
Resampling	BootstrapSplit, ShuffleSplit, RepeatedKFold, NestedCV	Repeated performance estimation and hyperparameter tuning

Research impact statement

DataSplits.jl has been developed to support benchmarking and validation workflows in scientific machine learning where split construction directly affects reported performance estimates. The package underpins a companion study currently under review (Crucitti & Mosquera Orgueira, 2026), which demonstrates empirically that split strategy choice can dominate differences between modelling algorithms in structured chemical datasets, with random splits systematically producing more optimistic performance estimates than distance-based and group-aware alternatives. A reproducible tutorial (notebooks/tutorial.jl) included in the repository provides side-by-side comparisons of random, Kennard–Stone, SPXY, and group-aware splits on controlled datasets, illustrating how coverage of feature and response spaces differs across strategies and quantifying group leakage under random versus group-aware partitioning. The tutorial also generates Figure 1. A benchmark suite (benchmark/benchmarks.jl) tracks runtime scaling of all strategies, including lazy variants, to support informed strategy selection on datasets of varying size. DataSplits.jl will be maintained by the GrHeCo-Xen group through the public GitHub repository, where users can report issues, request features, and contribute new splitting strategies.

AI usage disclosure

Generative AI tools, including Claude Code (Anthropic), were used to assist with software development — specifically in the integration of some algorithms — documentation drafting, and drafting and language editing of this paper. AI tools were not used to make autonomous design, implementation, or scientific decisions. All AI-assisted code was thoroughly verified against independent reference implementations and covered by the test suite. All AI-assisted text was reviewed and edited by the authors, and technical claims were checked against the implementation, tests, documentation, and cited literature. The authors are responsible for the final content of the manuscript, code, and documentation.

Acknowledgements

This research project was made possible through the access granted by the Galician Supercomputing Center (CESGA) to its supercomputing infrastructure. The supercomputer FinisTerae III and its permanent data storage system have been funded by the NextGeneration EU 2021 Recovery, Transformation and Resilience Plan, ICT2021-006904, and also from the Pluriregional Operational Programme of Spain 2014-2020 of the European Regional Development Fund (ERDF), ICTS-2019-02-CESGA-3, and from the State Programme for the Promotion of Scientific and Technical Research of Excellence of the State Plan for Scientific and Technical Research and Innovation 2013-2016 State subprogramme for scientific and technical infrastructures and equipment of ERDF, CESG15-DE-3114.

References

- Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213. <https://doi.org/10.1016/j.ins.2011.12.028>
- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>
- Blaom, A., Kiraly, F., Lienart, T., Simillides, Y., Arenas, D., & Vollmer, S. (2020). MLJ: A Julia package for composable machine learning. *Journal of Open Source Software*, 5(55), 2704. <https://doi.org/10.21105/JOSS.02704>
- Burns, J. W., Spiekermann, K. A., Bhattacharjee, H., Vlachos, D. G., & Green, W. H. (2023).

- Machine Learning Validation via Rational Dataset Sampling with *astartes*. *Journal of Open Source Software*, 8(91), 5996. <https://doi.org/10.21105/JOSS.05996>
- Clark, R. D. (1997). OptiSim: An Extended Dissimilarity Selection Method for Finding Diverse Representative Subsets†. *Journal of Chemical Information and Computer Sciences*, 37(6), 1181–1188. <https://doi.org/10.1021/CI970282V>
- Crucitti, D., & Mosquera Orgueira, A. (2026). Train-test splits matter more for evaluation than for performance. *Journal of Cheminformatics*.
- Ezenarro, J. J., & others. (2025). A novel method for the selection of calibration samples in near-infrared spectroscopy based on the onion algorithm: The XY-onion method. *Analytica Chimica Acta*, 344229. <https://doi.org/10.1016/j.aca.2025.344229>
- Gallagher, N., & O'Sullivan, D. (2020). *Selection of Representative Learning and Test Sets Using the Onion Method*. <https://doi.org/10.13140/RG.2.2.25372.67202>
- Galvão, R. K. H., Araújo, C., José, G. E., Pontes, M. J. C., Silva, E. C., & Saldanha, T. C. B. (2005). A method for calibration and validation subset partitioning. *Talanta*, 67(4), 736–740. <https://doi.org/10.1016/J.TALANTA.2005.03.025>
- He, Z., Sun, J., Zhang, Y., & Zhang, X. (2026). Field Strength Distribution-Based Sample Selection Method. *Journal of Chemometrics*, 40(1), e70094. <https://doi.org/10.1002/cem.70094>
- Innes, M. (2018). Flux: Elegant machine learning with Julia. *Journal of Open Source Software*, 3(25), 602. <https://doi.org/10.21105/JOSS.00602>
- JuliaML. (2022). *MLUtils.jl: Machine learning utility functions for julia*. <https://github.com/JuliaML/MLUtils.jl>
- Kennard, R. W., & Stone, L. A. (1969). Computer Aided Design of Experiments. *Technometrics*, 11(1), 137–148. <https://doi.org/10.1080/00401706.1969.10490666>
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*, 1137–1143. <https://doi.org/10.5555/1643031.1643047>
- Lelis Medeiros de Moraes, C. de, Santos, M. C. D., Lima, K. M. G., & Martin, F. L. (2019). Improving data splitting for classification applications in spectrochemical analyses employing a random-mutation Kennard-Stone algorithm approach. *Bioinformatics*, 35(24), 5257–5263. <https://doi.org/10.1093/BIOINFORMATICS/BTZ421>
- López de Prado, M. M. (2018). *Advances in financial machine learning*. Wiley. ISBN: 978-1-119-48211-6
- Næs, T., Isaksson, T., Fearn, T., & Davies, T. (2002). *A user-friendly guide to multivariate calibration and classification*. NIR Publications. <https://doi.org/10.1255/978-1-906715-25-0>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2018). *Scikit-learn: Machine Learning in Python*. <https://doi.org/10.48550/arXiv.1201.0490>
- Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillerá-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., & Dormann, C. F. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8), 913–929. <https://doi.org/10.1111/ECOG.02881>
- Sapto, A., O. Tade, M., & Vuthaluru, H. (2012). A Modified Kennard-Stone Algorithm for Optimal Division of Data for Developing Artificial Neural Network Models. *Chemical*

220 *Product and Process Modeling*, 7(1). <https://doi.org/10.1515/1934-2659.1645>

221 Snee, R. D. (1977). Validation of Regression Models: Methods and Examples. *Technometrics*,
222 19(4), 415–428. <https://doi.org/10.1080/00401706.1977.10489581>

DRAFT